Упражнение 3 Моделиране на обекти

Упражнение 3 има за цел да направим една от първите стъпки към рендиране на изображения – моделиране на обекти. За целта ще се фокусираме върху няколко основни момента:

- Първи стъпки в GLSL за работа с шейдъри в процеса на рендирането на изображението;
- Оцветяване на върхове;
- Примитивни типове за изчертаване;

1. Първи стъпки в GLSL

GLSL (OpenGL Shading Language) е език, базиран на C, който се използва за програмиране на шейдъри – малки програми, които се изпълняват на графичния процесор (GPU) и контролират как се изобразяват върховете и пикселите на екрана.

GLSL се използва в различни видове шейдъри, като най-често срещаните са:

- Vertex Shader – обработва върховете на примитивите.
- Fragment Shader – изчислява цвета на всеки пиксел.

Допълнителни видове шейдъри в OpenGL:

- Geometry Shader – може да променя броя на върховете.
- Tessellation Shaders – контролира детайлността на модели.
- Compute Shader – изпълнява общи изчисления на GPU.

GLSL поддържа различни типове данни, подобно на езиците C/C++, но с някои специфични разлики. Някои типове:

Тип	Описание	Пример за използване

void	За функции, които не връщат стойност	int count = 10;
		float scale = 1.5;
		bool isEnabled = true;

int	Цяло число (32-бита)	
uint	unsigned integer	
float	Число с плаваща запетая (32-бита)	
double	Число с плаваща запетая (64-бита)	
bool	Булева стойност (true или false)	

vec2	Двумерен вектор (x, y)	vec3 position = vec3(1.0, 2.0, 3.0);
		vec4 color = vec4(1.0, 0.0, 0.0, 1.0);
		
		Достъп до компонентите на вектор:
		//Достъп до x координата
		float x = position.x;
		// Достъп до зеления канал

vec3	Тримерен вектор (x, y, z)	
vec4	Четиримерен вектор (x, y, z, w)	

		float g = color.g;
		// Извличане на първите 2 компонента
		vec2 xy = position.xy;

mat2	2x2 матрица	// Единична матрица
		mat4 model = mat4(1.0);
		vec4 transformedPos = model * vec4(position, 1.0);

mat3	3x3 матрица	
mat4	4x4 матрица (често използвана за 3D трансформации)	

GLSL поддържа масиви, но те трябва да имат фиксиран размер. Пример:

```
float values[5];

vec3 colors[3] = vec3[](vec3(1,0,0), vec3(0,1,0), vec3(0,0,1)); // Червено, зелено, синьо
```

GLSL използва специални квалификатори за входни и изходни променливи между шейдърите.

Квалификатор	Описание	Пример за използване
in	Използва се за входни данни в шейдърите	in vec3 position;
out	Определя изходна стойност	out vec4 FragColor;
uniform	Uniform променливи (глобални за всички върхове/фрагменти). Използват се за предаване на константни стойности като матрици и осветление.	uniform mat4 viewMatrix;

layout(location = X) in	Използва се във Vertex Shader, за да зададе конкретен входен атрибут към върха (vertex). Това позволява на OpenGL да знае от кой входен индекс да прочете дадена информация в Vertex Array Object (VAO).	// Позиция на върха
		layout(location = 0) in vec3 aPos;
		// Цвят на върха
		layout(location = 1) in vec3 aColor;
		// Текстурни координати
		layout(location = 2) in vec2 aTexCoord;

Примерен Vertex Shader (ще бъде допълнен в подобен вид в следващите упражнения)

```
#version 460 core  // Дефинира версията на GLSL

layout (location = 0) in vec3 aPos; // Входна променлива (позиция на върха)
layout (location = 1) in vec3 aColor; // Входна променлива (цвят)

out vec3 vertexColor; // Изходна променлива за фрагментния шейдър

uniform mat4 model; // Матрица за трансформация
uniform mat4 view;
uniform mat4 projection;
```

```
void main() {
   gl_Position = projection * view * model * vec4(aPos, 1.0);
   vertexColor = aColor;
}
```

Примерен Fragment Shader

```
#version 460 core

in vec3 vertexColor; // Входна променлива от Vertex Shader
out vec4 FragColor;  // Изходен цвят

void main() {
   FragColor = vec4(vertexColor, 1.0);
}
```

В GLSL има специални вградени променливи, които OpenGL използва автоматично за позициониране. На този етап сме се сблъсквали с gl_Position. В GLSL, gl_Position е задължителна изходна променлива във Vertex Shader. Тя определя крайната позиция на върха (vertex) в клип пространството (clip space). След като Vertex Shader изчисли gl_Position, OpenGL използва тази стойност, за да определи къде върхът ще бъде изобразен на екрана.

gl_Position е четиримерен вектор vec4(x, y, z, w). x, y, z са координати на върха, а w – четвъртата хомогенна координата (perspective division) (обикновено 1.0). Ако w != 1.0, тогава всички координати се делят на w, което създава перспектива.

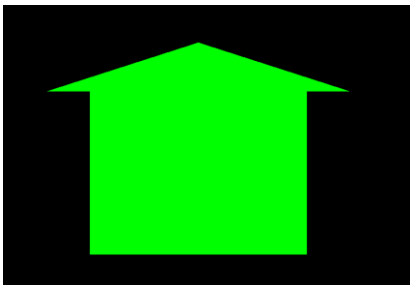
Документация на GLSL.

2. Оцветяване на върхове

В първоначалния проект използвахме фрагментния шейдър, за да зададем оцветяването на рендирания обект.

```
void main() {
    FragColor = vec4(0.0, 1.0, 0.0, 1.0);
}
```

При този подход всички пиксели на обекта получават един и същ цвят, което ограничава възможностите за по-сложни визуални ефекти. Не може да се реализира интерполация на цветове между върховете (гладък цветови преход), нито да се използват динамични материали, текстури или интерактивни промени в оцветяването.



По-гъвкав подход е цветът да бъде дефиниран още на етапа на задаване на върховете (в съответните класове от пакета demo.assets). По този начин към всеки връх могат да бъдат асоциирани допълнителни атрибути, включително информация за цвят.

Нека основата и покривът на изобразената къщичка бъдат оцветени в различни цветове. Например, основата може да бъде в сив цвят. В този случай цветът се задава чрез тройка стойности, представящи RGB компонентите. Стойностите на всеки цветови канал се дефинират в нормализирания диапазон от 0 до 1.

По отношение на основата на къщичката към буфера ще бъдат подадени следните стойности (клас RectangleGeometry):

```
private float[] createVertices() {
    return new float[]{
        // x y z          r g b
        -0.5f, -0.7f, 0f, 0.5f, 0.5f, 0.5f,
        0.5f, -0.7f, 0f, 0.5f, 0.5f, 0.5f,
        0.5f, 0.3f, 0f, 0.5f, 0.5f, 0.5f,

        -0.5f, -0.7f, 0f, 0.5f, 0.5f, 0.5f,
        0.5f, 0.3f, 0f, 0.5f, 0.5f, 0.5f,
        -0.5f, 0.3f, 0f, 0.5f, 0.5f, 0.5f
    };
}
```

Първите 3 подавани стойности са координати (X, Y, Z), докато следващите 3 числа са цветови стойности (R, G, B) в диапазона [0.0, 1.0]. В случая по отношение на всички описани върхове са подадени данни за един и същи цвят (0.5, 0.5, 0.5).

Следва да дефинираме начина, по който данните ще бъдат интерпретирани при изчитане от буфера (VBO). За целта е необходимо да добавим втори атрибут, който да бъде асоциран с информацията за цвета на всеки връх. Така в класа RectangleGeometry трябва да се дефинират два атрибута на върха:

- първият атрибут (location = 0) – за позицията на върха (x, y, z);
- вторият атрибут (location = 1) – за цвета на върха (r, g, b).

```
setAttributes(new VertexAttribute[]{
        new VertexAttribute(0, 3),     // позиция (x, y, z)
        new VertexAttribute(1, 3),     // цвят (r, g, b)
  });
  setStride(6 * Float.BYTES);      // 3 за позиция + 3 за цвят
```

Стойността на stride определя общия размер (в байтове) на данните, асоциирани с един връх. В конкретния случай всеки връх съдържа 6 float стойности — 3 за позиция и 3 за цвят, поради което се използва 6 * Float.BYTES.

Нека при покрива на къщичката зададем различни цветове на върховете, които го описват (клас TriangleGeometry).

```
private float[] createVertices() {
  return new float[]{
        // x y z              r g b
        -0.7f, 0.3f, 0.0f, 1.0f, 0.0f, 0.0f,
        0.7f, 0.3f, 0.0f, 0.0f, 1.0f, 0.0f,
        0.0f, 0.6f, 0.0f,  0.0f, 0.0f, 1.0f
  };
}
```

Аналогично дефинираме втори атрибут, асоциран с данните за цветовете.

```
setAttributes(new VertexAttribute[]{
        new VertexAttribute(0, 3),     // позиция (x, y, z)
        new VertexAttribute(1, 3),     // цвят (r, g, b)
  });
  setStride(6 * Float.BYTES);      // 3 за позиция + 3 за цвят
```

При това положение шейдърите ще имат следния вид:

Vertex шейдър

```
#version 460 core
layout(location = 0) in vec3 aPos;   // Позиция на върха
```

```
layout(location = 1) in vec3 aColor; // Цвят на върха


out vec3 vertexColor; // Изход за фрагментния шейдър


void main() {
   gl_Position = vec4(aPos, 1.0);
   vertexColor = aColor; // Предаване на цвета към фрагментния шейдър
}
```

Фрагментен шейдър:

```
#version 460 core
in vec3 vertexColor;  // Получаваме интерполирания цвят от vertex шейдъра
out vec4 FragColor;


void main() {
   FragColor = vec4(vertexColor, 1.0); // Оцветяване с интерполиран цвят
}
```

Крайният резултат от изменения код ще бъде оцветената къщичка.

3. Примитивни типове за изчертаване

При рендирането на графика в OpenGL всички обекти се изграждат от графични примитиви. Те представляват основните градивни блокове за рисуване на геометрични фигури, като точки, линии и триъгълници. Чрез комбинация от тези примитиви можем да създаваме сложни 2D и 3D обекти. Всеки примитив се състои от един или повече върхове (vertices), които определят неговата форма.

OpenGL поддържа няколко вида примитиви, които могат да бъдат използвани за различни графични приложения.

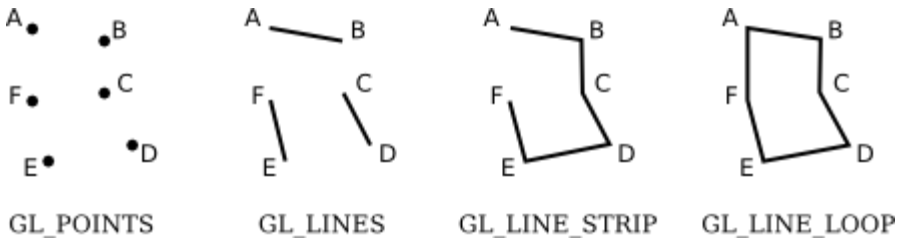
- **Точки (GL_POINTS)**

  Всеки връх се изобразява като отделна точка на екрана. Използва се за ефекти като звезди, частици и точкови облаци.

- **Линии (GL_LINES, GL_LINE_STRIP, GL_LINE_LOOP)**

1. **GL_LINES** – всяка двойка върхове формира отделна линия.
2. **GL_LINE_STRIP** – последователно свързани върхове без затваряне.
3. **GL_LINE_LOOP** – като GL_LINE_STRIP, но свързва последния връх с първия.

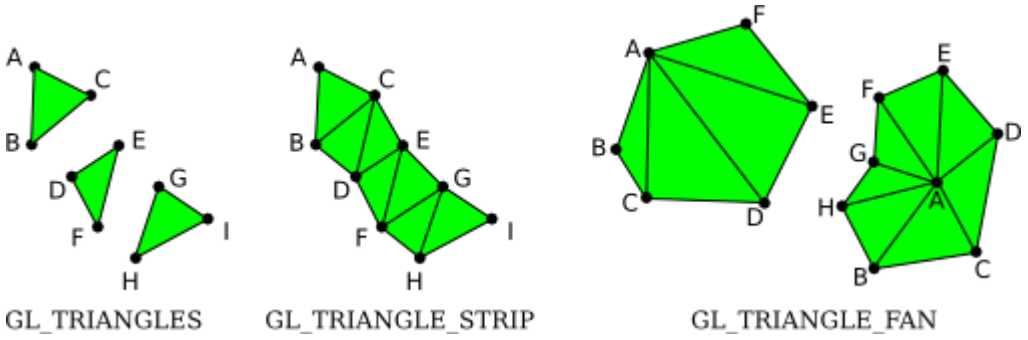
Използват се за контури, мрежи и схеми.



- **Триъгълници (GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN)**

1. **GL_TRIANGLES** – всеки три върха формират отделен триъгълник.
2. **GL_TRIANGLE_STRIP** – последователно свързани триъгълници, споделящи върхове. Всеки нов връх образува нов триъгълник със двата предходни
3. **GL_TRIANGLE_FAN** – всички триъгълници са свързани с общ център.

Използват се за изграждане на сложни 3D обекти.



Примери:

GL_LINES

Създайте аналогичен на TriangleGeometry клас за геометрия – PrimitiveGeometry. Задайте следните данни:

```
private float[] createVertices() {
    return new float[]{
        // X, Y, Z,  R, G, B
        -0.5f, -0.2f, 0.0f,  1.0f, 0.0f, 0.0f,
```

```
            0.0f, -0.5f, 0.0f,  0.5f, 0.5f, 0.0f,
            0.5f, -0.3f, 0.0f,  0.0f, 1.0f, 0.0f,
            0.4f,  0.0f, 0.0f,  0.0f, 0.5f, 0.5f,
        };
    }
```

Създайте клас PrimitiveDemo за обект от сцена (аналогичен на House), като използвате като примитив линия.

```
    private void buildPrimitive() {
        Mesh primitiveMesh = new Mesh(DrawMode.LINES, new PrimitiveGeometry());
        ModelElement primitive = new ModelElement(primitiveMesh);
        getModel().addElement(primitive);
    }
```

Създайте сцена с новосъздадения обект в класа SceneBuilder и стартирайте проекта.

```
        Scene scene = new Scene();
        SceneObject primitiveDemo = new PrimitiveDemo();
        scene.addSceneObject(primitiveDemo);
```

GL_LINE_STRIP

Заменете използвания примитив с GL_LINE_STRIP и стартирайте проекта.

```
    private void buildPrimitive() {
        Mesh primitiveMesh = new Mesh(DrawMode.LINE_STRIP, new LineGeometry());
        ModelElement primitive = new ModelElement(primitiveMesh);
        getModel().addElement(primitive);
    }
```

GL_LINE_LOOP

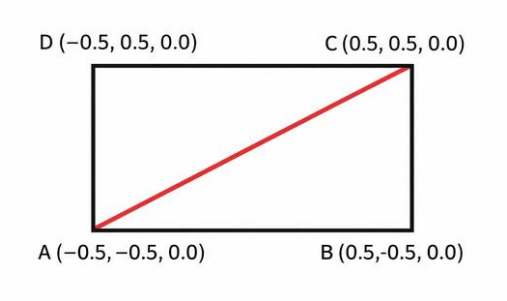
Заменете използвания примитив с GL_LINE_LOOP и стартирайте проекта.

```
    private void buildPrimitive() {
```

```
        Mesh primitiveMesh = new Mesh(DrawMode.LINE_LOOP, new LineGeometry());
        ModelElement primitive = new ModelElement(primitiveMesh);
        getModel().addElement(primitive);
    }
```

GL_TRIANGLES

Ако искаме да изрисуваме правоъгълник, той трябва да бъде описан от 2 триъгълника – ABC и ACD (2*3 = 6 върха).




```
    private float[] createVertices() {
        return new float[]{
            // x     y     z     r     g     b
            -0.5f, -0.5f, 0f,  1f, 0f, 0f,   // A
            0.5f, -0.5f, 0f,   0f, 1f, 0f,   // B
            0.5f,  0.5f, 0f,   0f, 0f, 1f,   // C
            -0.5f, -0.5f, 0f,   1f, 0f, 0f,   // A
            0.5f,  0.5f, 0f,   0f, 0f, 1f,   // C
            -0.5f,  0.5f, 0f,   1f, 1f, 0f    // D
        };
    }
```

```
    private void buildPrimitive() {
        Mesh primitiveMesh = new Mesh(DrawMode.TRIANGLES, new PrimitiveGeometry());
        ModelElement primitive = new ModelElement(primitiveMesh);
        getModel().addElement(primitive);
    }
```

GL_TRIANGLE_STRIP

С помощта на GL_TRIANGLE_STRIP можем да използваме 4 върха вместо 6. Последователността на върховете трябва да бъде такава, че да се получат триъгълниците:

- първи триъгълник – A B D
- втори триъгълник – B D C

Вторият и третият връх от първия триъгълник се преизползват като първи и втори връх от втория триъгълник.

```
private float[] createVertices() {
    return new float[]{
        // x     y     z     r    g    b
        -0.5f, -0.5f, 0f,   1f, 0f, 0f,  // A
         0.5f, -0.5f, 0f,   0f, 1f, 0f,  // B
        -0.5f,  0.5f, 0f,   1f, 1f, 0f,  // D
        0.5f,  0.5f, 0f,    0f, 0f, 1f   // C
    };
}
```

```
private void buildPrimitive() {
    Mesh primitiveMesh = new Mesh(DrawMode.TRIANGLE_STRIP, new PrimitiveGeometry());
    ModelElement primitive = new ModelElement(primitiveMesh);
    getModel().addElement(primitive);
}
```

GL_TRIANGLE_FAN

При GL_TRIANGLE_FAN първият връх е общ, а всички следващи върхове образуват триъгълници с него. Последният връх от първия триъгълник се преизползва като втори връх от следващия триъгълник.

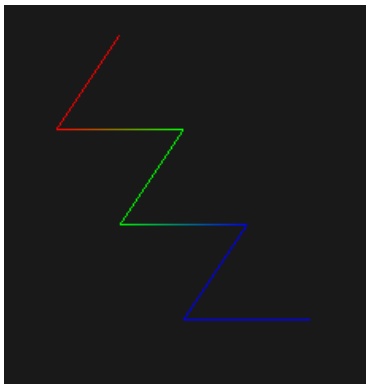
```
private float[] createVertices() {
    return new float[]{
        // x     y     z     r     g     b
        -0.5f, -0.5f, 0f,   1f, 0f, 0f,   // A (общ)
        0.5f, -0.5f, 0f,   0f, 1f, 0f,   // B
        0.5f,  0.5f, 0f,   0f, 0f, 1f,   // C
        -0.5f,  0.5f, 0f,   1f, 1f, 0f    // D
    };
}
```

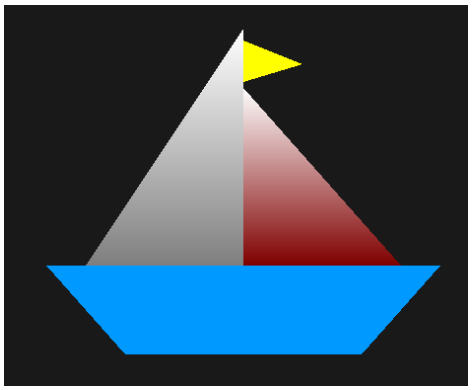
```
private void buildPrimitive() {
    Mesh primitiveMesh = new Mesh(DrawMode.TRIANGLE_FAN, new PrimitiveGeometry());
    ModelElement primitive = new ModelElement(primitiveMesh);
    getModel().addElement(primitive);
}
```

Задачи

1. Тествате примерите по-горе. Като основа можете да използвате: https://github.com/donikast/GraphicsSystems/tree/Lab3

2. Рендирайте следните изображения





3. Изобразете окръжност/кръг в цвят/цветове по избор.

В OpenGL (и съответно LWJGL) няма вграден примитив окръжност, така че трябва да я изградим ръчно с помощта на триъгълници или линии. Има два основни метода за изобразяване на окръжност:

- с контур (GL_LINE_LOOP) – окръжността се рисува като поредица от линии, свързани в кръг.
- със запълнена повърхност (GL_TRIANGLE_FAN) – използваме триъгълници, за да запълним окръжността.

За изчисляване на върховете можем да използваме следната логика:

1. Избираме център и радиус – определяме (cx, cy) като център и radius като радиус.
2. Разделяме окръжността на сегменти – например 100 сегмента за гладка окръжност.
3. Изчисляваме върховете – за всяка точка на окръжността изчисляваме:

$$x = cx + \cos(\theta) \times radius$$

$$y = cy + \sin(\theta) \times radius$$

   където θ е ъгълът, разделящ всеки сегмент.

4. Изпращаме координатите в OpenGL и използваме GL_TRIANGLE_FAN, за да ги свържем.

Примерна реализация с център с координати (0.0, 0.0, 0.0):

В класа на геометрията:

```
private int numSegments = 100; // Колко сегмента да има окръжността
private float radius = 0.5f;
private float centerX = 0.0f;
private float centerY = 0.0f;
```

Определяне на координатите:

```
private float[] createVertices() {
    float[] data = new float[(numSegments + 2) * 6];
    data[0] = centerX;
    data[1] = centerY;
    data[2] = 0.0f;
    data[3] = 1.0f;  //R
    data[4] = 1.0f;  //G
    data[5] = 1.0f;  //B
```

```
    for (int i = 1; i <= numSegments + 1; i++) {
        float angle = (float) (2.0 * Math.PI * i / numSegments);
        data[i * 6] = centerX + (float) Math.cos(angle) * radius;
        data[i * 6 + 1] = centerY + (float) Math.sin(angle) * radius;
        data[i * 6 + 2] = 0.0f;
        data[i * 6 + 3] = 1.0f;
        data[i * 6 + 4] = 0.0f;
        data[i * 6 + 5] = 1.0f;
    }
    return data;
}
```

Задачи за допълнителни упражнения

1. Рендирайте следното изображение /бял правоъгълник с дупка в средата/:

2. Нарисувайте слънце с лъчи